

Generador Académico RAG

Un sistema de IA generativa que sintetiza artefactos académicos personalizados anclados en el historial real del grupo y el plan de estudios, mediante un pipeline RAG construido de punta a punta.

GRUPO 14	Martin, Gentil, Natalichio, Cuenca, Ocampo, Aubert
PIPELINE	pdfplumber → SQLite + Chroma (MiniLM) → qwen2.5:14B (Ollama)
ENTORNO	híbrido: pipeline local · generación en GPU (Colab)

01 Planificación y metodología

El objetivo es construir un sistema de IA generativa que, alimentado por **dos bases de datos** —una **relacional** con el historial académico del grupo y las correlatividades, y una **vectorial** con la documentación de la carrera— **genere artefactos académicos personalizados**: planes de cursada justificados, informes de trayectoria, cartas de pasantía, recomendaciones de orientación, diagnóstico grupal y simulaciones what-if.

El reencuadre central del proyecto fue pasar de un *asistente de preguntas* (que un SQL o NotebookLM resolverían) a un **generador**: cada salida es texto nuevo sintetizado, con razonamiento y estilo, no un campo de una tabla. Un SQL responde qué materias *podés* cursar; este sistema redacta qué *conviene* y por qué, cruzando notas, correlativas y patrón de rendimiento.

La metodología siguió el pipeline de Ciencia de Datos completo: ingesta y preprocesamiento de un corpus mixto, **persistencia híbrida** (relacional para lo tabular, vectorial para la documentación), análisis exploratorio, visualización, clustering, generación con control de tono y evaluación cuantitativa del anclaje (grounding).

413+136

filas (relacional) + docs
(vectorial)

36

correlatividades parseadas

6

generadores de artefactos

3

tonos controlables

02 Datos, EDA y preprocesamiento

El corpus combina lo **estructurado** (notas, años y correlatividades) con lo **no estructurado** (chunks del plan de estudios), y **cada parte se persiste en la base que le corresponde**: lo tabular en una base **relacional** (SQLite), la documentación en una base **vectorial** (Chroma). Las fuentes son PDFs reales: los estados académicos y exports de exámenes de los **seis integrantes del grupo**, el Plan de Sistemas 2023 (correlatividades del Anexo I y equivalencias del Anexo II) y el **Diseño Curricular completo (Ord. 1877)**, del que se extraen las fichas de las 36 materias con sus contenidos mínimos.

El preprocesamiento crítico fue el **join entre el estado académico y el plan**: los códigos institucionales no coinciden entre documentos, así que la clave de unión es el *nombre de materia normalizado* (minúsculas, sin tildes ni puntuación). Se parsearon las 36 obligatorias con sus correlativas, resolviendo casos borde como las integradoras, cuyo nombre se parte en dos líneas. El parseo produce 549 registros que se reparten entre las dos bases: **413 filas a la relacional** (377 de estados académicos + 36 de correlatividades) y **136 documentos a la vectorial** (100 chunks de prosa del diseño curricular + 36 fichas de materia con sus contenidos mínimos, del Anexo de la Ord. 1877). No se aplicó lematización ni stemming clásicos sobre la prosa: el texto del plan se indexa con **embeddings de MiniLM**, que operan a nivel sub-palabra, así que el preprocesamiento que sí importa no es morfológico sino el *chunking* de la prosa y la normalización de nombres para el join.

El historial de notas de exámenes usa los nombres del **Plan 2008**, mientras que el estado académico ya migró a los del **Plan 2023**. Para que el cruce no pierda notas aprobadas por equivalencia, incorporamos el **Anexo II** del plan (régimen de equivalencias 2008↔2023) como tabla de canonización de nombres: toda materia se lleva a su nombre 2023 antes del join, con tolerancia a la truncación de nombres largos del PDF.

El export de exámenes resultó además **más completo que el estado académico**: contiene materias aprobadas (con su nota) que ni siquiera figuran como fila en el estado. Esas materias se incorporan al corpus como aprobadas, evitando que el planificador las recomiende como pendientes. En paralelo, se distingue el estado real de cada materia — aprobada, en curso («cursando», aún sin nota) o no iniciada— para que el plan de cursada excluya lo que ya se está cursando y ofrezca, además de las obligatorias habilitadas, las electivas del plan todavía disponibles.

El análisis exploratorio trabaja sobre **221 registros reales** de notas (una entrada por materia aprobada con calificación numérica), correspondientes a los seis integrantes. Los promedios van de **8.0 a 8.7**, un grupo parejo y de buen rendimiento; las diferencias aparecen al desagregar por área temática, que es lo que explota el diagnóstico grupal.

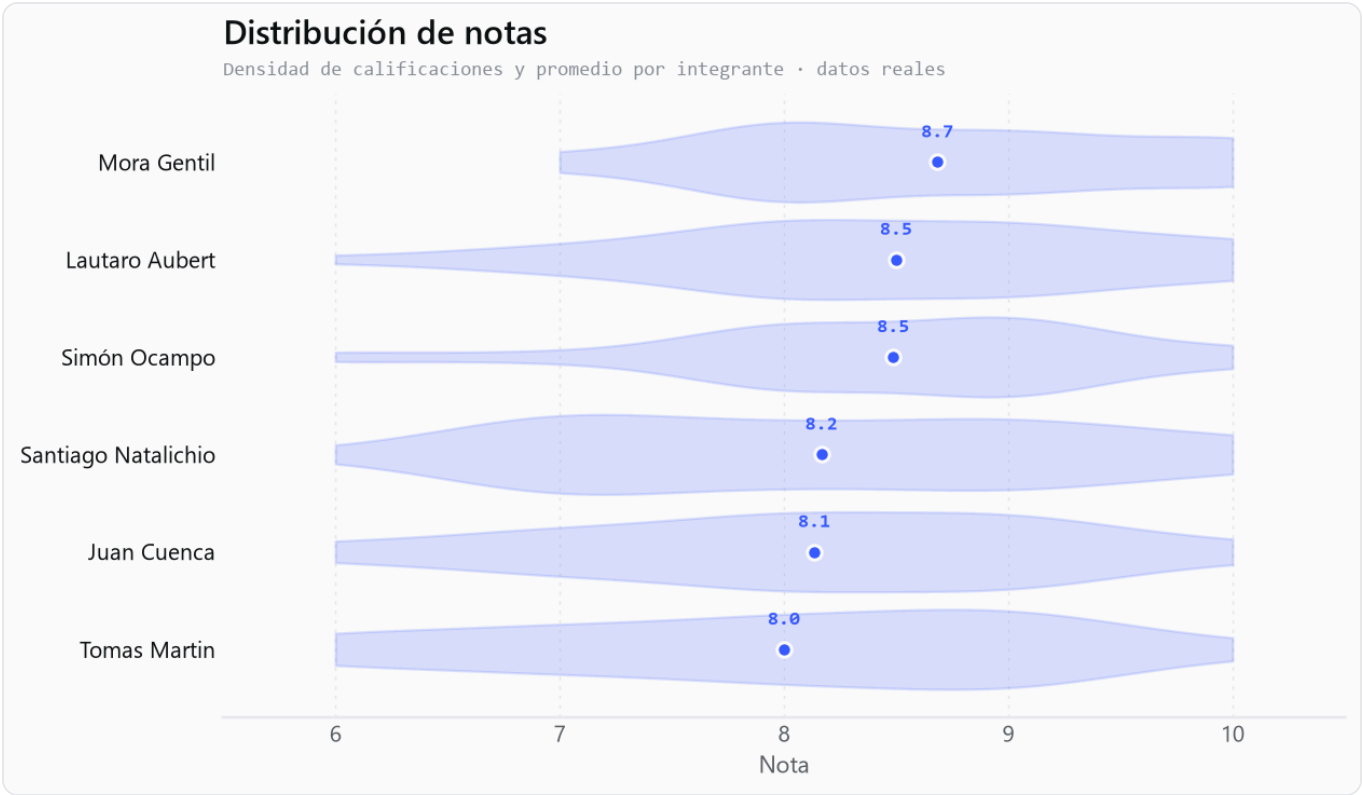


Fig. 1 – Distribución de notas por integrante (datos reales de los seis): promedios entre 8.0 y 8.7.

Avance académico por año

Materias aprobadas por integrante en cada nivel • datos reales

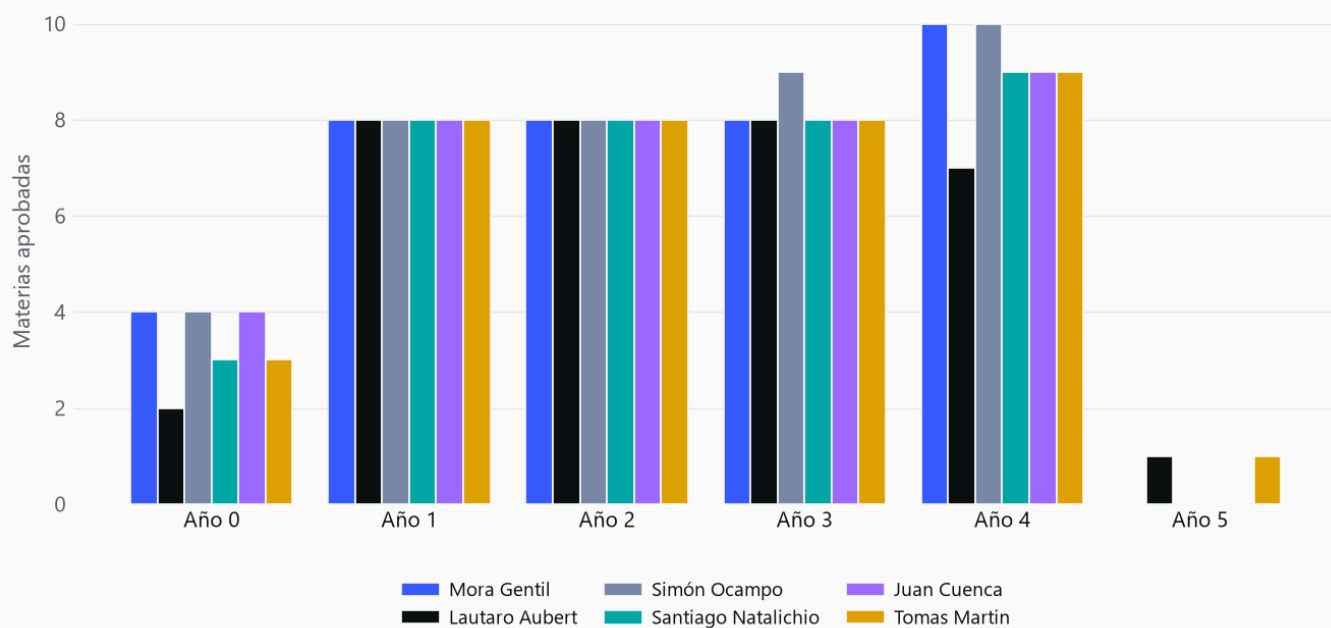


Fig. 2 – Avance académico real por año: materias aprobadas por integrante.

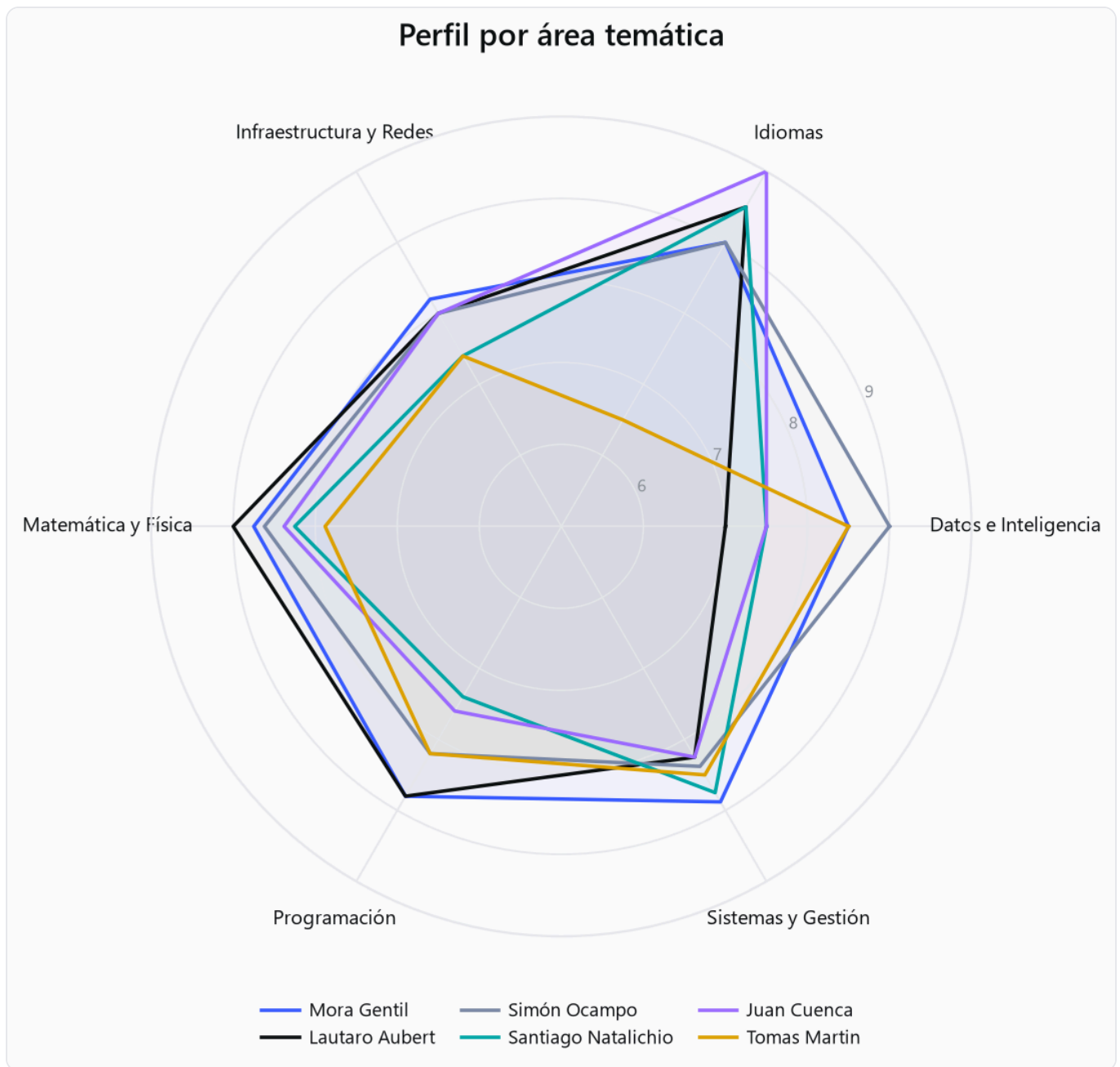


Fig. 3 – Perfil por área temática. Las 36 obligatorias se agrupan en 6 áreas para comparar fortalezas.

03 Pipeline RAG e implementación

El pipeline se construyó en módulos independientes, cada uno con self-checks:

```
extract.py      PDF → texto (pdfplumber)
documents.py    texto → documentos + correlatividades; normalizar() = clave de join
db.py           datos tabulares → SQLite (relacional); consultas SQL (historial, correlati
ingest.py       prosa del plan + fichas de materia (contenidos) → Chroma (vectorial); busc
rag.py          retrieval híbrido (SQL + vector) → LLM (qwen2.5:14B vía Ollama); fallback
 analisis.py    DataFrame de notas reales (6 integrantes, desde SQLite) + agregaciones por
 figuras.py     EDA, radar, clustering (KMeans/PCA), t-SNE de embeddings
 generar.py     6 generadores de artefactos con control de tono
 evaluar.py     grounding léxico + semántico + comparar_con_sin_rag()
```

Persistencia híbrida: cada dato en su base

El historial académico y las correlatividades son **datos tabulares** (alumno, materia, nota, año; Nº de plan y correlativas): registros que se consultan con **exactitud** —filtros, joins, agregaciones—, no con similitud semántica. Por eso viven en una base **relacional** (SQLite, db.py). La **documentación del plan** —prosa larga del régimen, perfil y alcances— va en la base **vectorial** (Chroma), donde el matching por embeddings hace lo que un SELECT no puede. Meter lo tabular en la vectorial habría sido usar búsqueda aproximada para un lookup determinístico; la decisión de separarlos es deliberada y mejora exactitud y claridad.

La vectorización usa **all-MiniLM-L6-v2** (el embedding por defecto de Chroma, local y offline). El retrieval es **híbrido** y **combina las dos bases: relacional** (consultas SQL que anclan exactamente las materias aprobadas de un alumno y las correlatividades) + **vectorial** (similitud de embeddings sobre la prosa del plan). Una misma consulta cruza ambas en un solo contexto. La generación corre sobre **qwen2.5:14B vía Ollama**; si Ollama está apagado, la parte RAG sigue funcionando y devuelve el contexto recuperado.

Arquitectura de ejecución híbrida

Todo el pipeline RAG —extracción, base relacional, embeddings, base vectorial y recuperación— corre **local** en la máquina del grupo. Lo único que cambia de lugar es la **generación del LLM**: como `rag.py` consume Ollama por HTTP (variable OLLAMA_URL), el modelo puede correr en la misma PC (CPU) o en una **GPU T4 gratuita de Google Colab**, expuesta por un túnel de URL fija. Esto permite usar un modelo grande (14B) con respuestas en segundos sin GPU propia, sin cambiar una línea del pipeline. Es una decisión de despliegue: en producción el sistema corre 100% local y offline; para la demo se aprovecha la GPU remota. Los prompts nunca pasan por un proveedor de inferencia de terceros —corren en una VM efímera del propio grupo—, preservando la privacidad de los datos académicos.

Estrategia de prompting

El sistema instruye al modelo a usar **exclusivamente** el contexto recuperado y a no inventar notas, materias ni correlatividades, pero también a **interpretar y seleccionar**: elegir los pocos datos relevantes y explicar qué revelan, en lugar de enumerar todas las notas. Cada artefacto define una tarea concreta y una estructura esperada (por ejemplo, la carta de pasantía exige primera persona, conectar fortalezas con competencias del puesto y no listar el historial). El **control de tono** (técnico, motivacional, honesto) reescribe el mismo dato en distinto registro: es lo que hace al sistema generativo y no un reporte fijo.

"Trabajás solo con los datos del contexto: no inventes ni supongas. Pero no sos un loro de datos: interpretá y seleccioná los datos relevantes y explicá qué revelan; nunca enumeres todas las notas."

04 Clustering y espacio semántico

Sobre el perfil por área (estandarizado) se aplicó **KMeans** y se proyectó a 2D con PCA para visualizar grupos de perfiles afines. Por separado, se proyectaron las 36 obligatorias con **t-SNE sobre sus embeddings**: las materias de la misma área caen juntas, lo que valida que el espacio semántico capturó la estructura del plan, condición necesaria para que el recomendador por matching semántico funcione.

Clustering de perfiles académicos

KMeans (k=3) sobre el perfil por área, proyectado con PCA

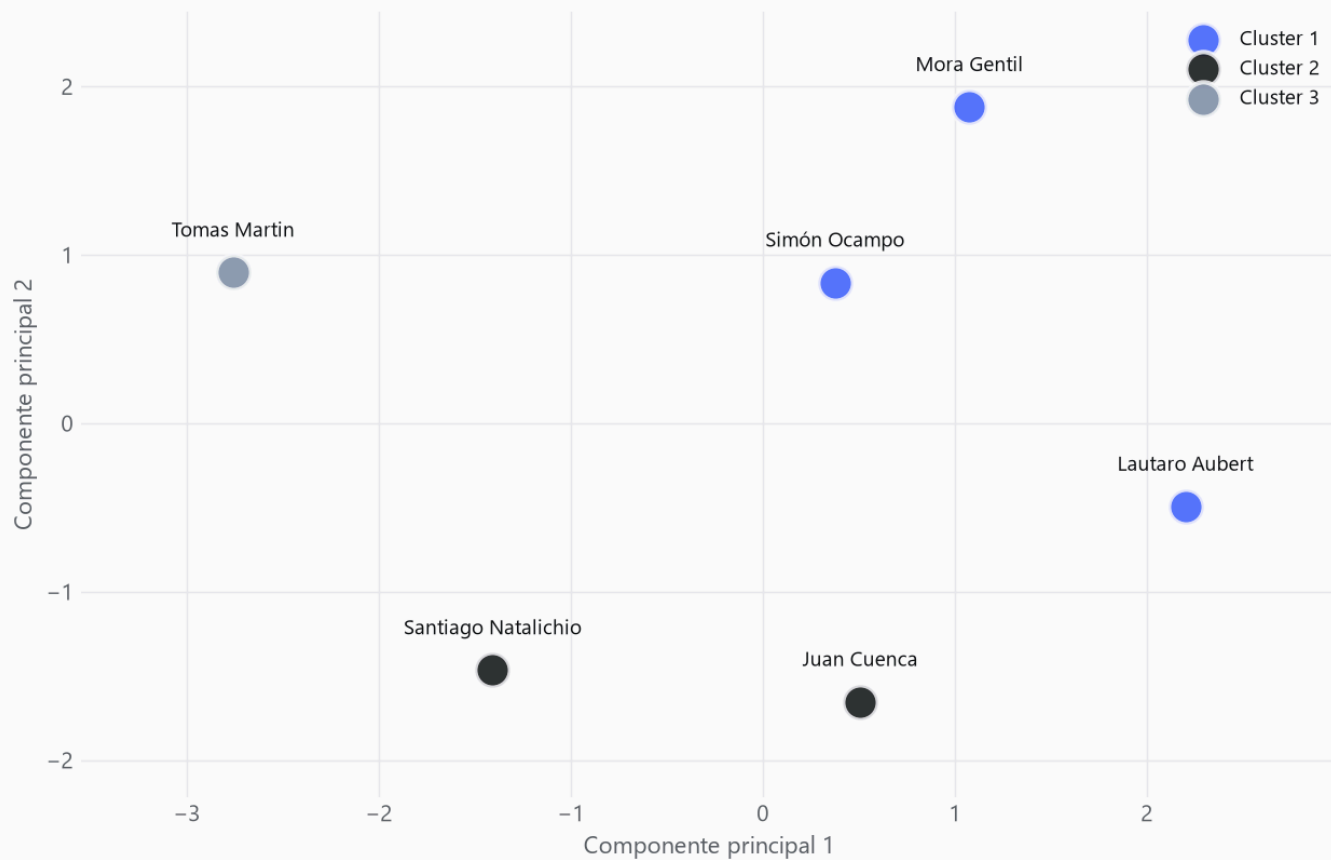


Fig. 4 – Clustering de perfiles académicos (KMeans sobre perfil por área, proyección PCA).

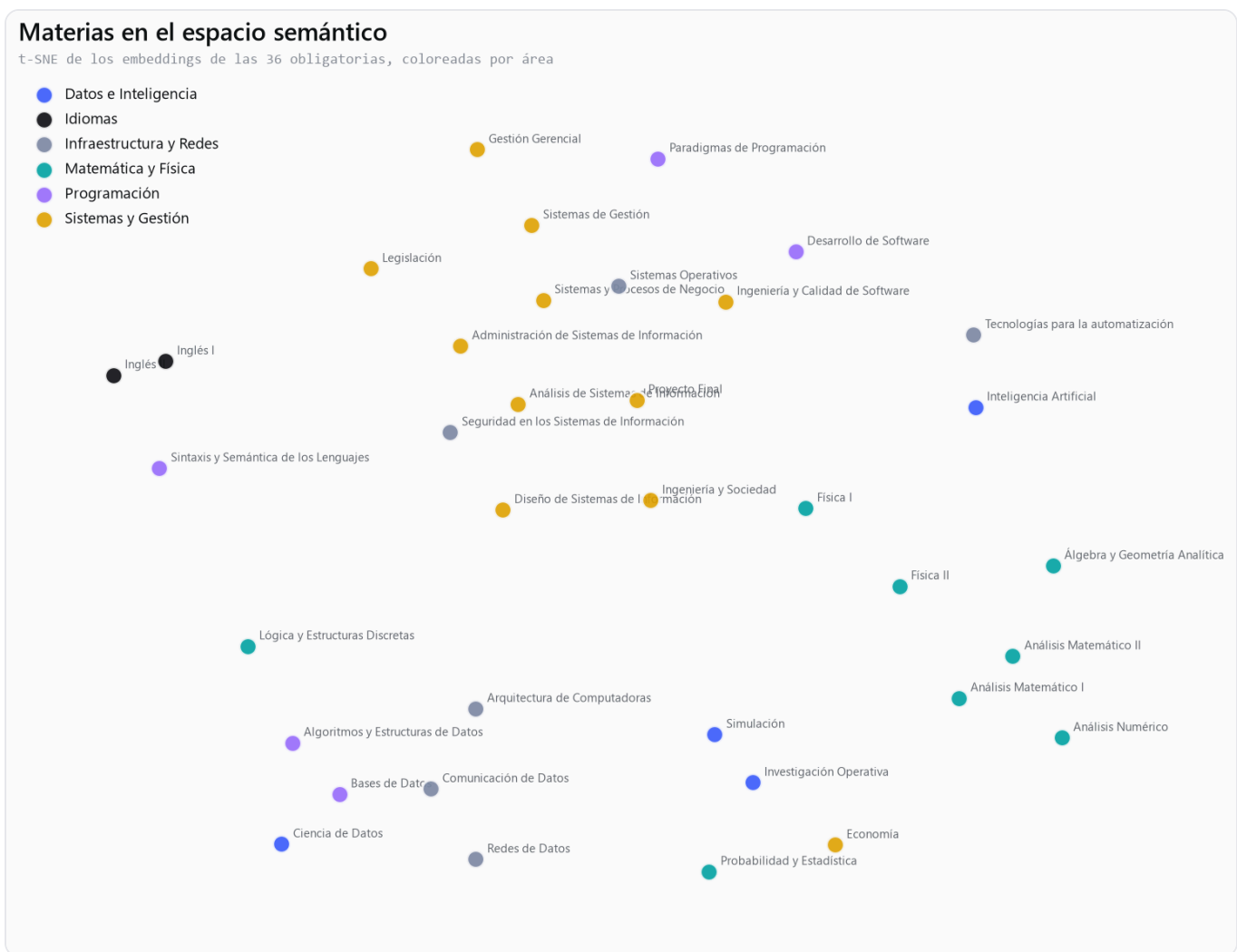


Fig. 5 – Las 36 obligatorias en el espacio semántico (t-SNE de embeddings), coloreadas por área.

05 Evaluación: grounding con/sin RAG

Qué medimos y por qué. El objetivo es cuantificar **cuánto del contexto inyectado usa el modelo para responder y cuánto resuelve con su propio conocimiento**. Eso es justo lo que captura el **grounding**: la fracción de la salida anclada en el contexto es lo que el modelo *tomó del RAG*; el complemento (1-grounding) es redacción o conocimiento propio. Y correr el mismo pedido **con** vs **sin** el contexto aísla el aporte del conocimiento privado: el salto entre ambos es cuánto pesó el RAG (Fig. 6).

Medimos el anclaje con **cuatro métricas sobre tres ejes**. **(a) Solapamiento:** el **grounding léxico** es la fracción de tokens informativos de la salida (materias, notas) presentes literalmente en el contexto —explicable, pero penaliza la paráfrasis— y el **grounding semántico** promedia el máximo coseno entre cada oración de la salida y los fragmentos del contexto (embeddings MiniLM), reconociendo la reformulación legítima. **(b) Corrección factual:** la **precisión factual** extrae los pares (materia, nota) que la salida afirma y verifica cada uno contra el registro real del alumno —mide acierto, no cercanía—. **(c) Juicio de un LLM:** la **faithfulness** usa el propio modelo como evaluador, puntuando de 0 a 1 cuánto se apoya la respuesta solo en el contexto.

La **demo estrella** compara el mismo pedido **con RAG** (contexto real) contra **sin RAG** (sin contexto). Promediando cuatro artefactos (plan de cursada, recomendación de orientación, informe de trayectoria y carta de pasantía) con **qwen2.5:14B**, el resultado más contundente es la **precisión factual**: sobre los artefactos que llegan a afirmar pares (materia, nota) verificables —en esta corrida, la carta de pasantía—, con RAG es **1.0** (el 100% de los pares coincide con el registro real: cero notas inventadas), mientras que sin RAG el modelo no afirma ningún dato verificable. La **faithfulness** juzgada por el LLM cae de **0.75 con RAG a 0.25 sin RAG**, y el grounding de **0.69 a 0.22** (léxico) y **0.60 a 0.53** (semántico). En las cuatro métricas, con RAG > sin RAG: la evidencia medible de que el conocimiento privado es el corazón del producto, no un adorno.

Vale una lectura crítica. El grounding léxico **premia la cita literal**: un artefacto profesional que parafrasea e interpreta (como la carta, que por diseño *no* enumera notas) obtiene un léxico más bajo aun siendo correcto —ronda 0.6, no 1—; por eso sumamos la precisión factual, que sí mide acierto. La **faithfulness** separa bien en promedio (0.75 vs 0.25), aunque por artefacto es más gruesa: en los más narrativos (informe, carta) puntúa 1.0 con RAG contra 0.0 sin, mientras que en plan y recomendación el juez fue conservador (0.5 en ambos). Como la generación es estocástica, el valor absoluto de una corrida tiene **varianza**; lo robusto es la **dirección** (con RAG > sin RAG en las cuatro métricas) y la precisión factual de 1.0. Limitación de fondo: ninguna métrica detecta *contradicciones* lógicas sutiles, solo cercanía, corrección de pares y un juicio agregado.

Calidad del retriever (recuperación, sin LLM)

Las métricas anteriores evalúan la generación; por separado medimos la **recuperación** sola. Sobre 12 consultas-tema redactadas *sin* nombrar la materia —para forzar el match semántico y no léxico— se verifica si la ficha de la materia esperada aparece en el top-5 de la base vectorial: **hit-rate@1 = 0.92**, **hit-rate@3 = 0.92** y **MRR = 0.92**. Estos números son fruto de una **decisión de preprocesamiento medida**: la ficha que se vectoriza contiene solo el *nombre* de la materia y sus *contenidos mínimos*, y descarta el preámbulo y la tabla de metadatos (carga horaria, bloque, competencias) que el PDF repite casi idéntica en las 36 fichas. Ese boilerplate compartido acercaba todas las fichas entre sí en el espacio de embeddings; quitarlo subió el hit-rate@1 de **0.50 a 0.92** y el MRR de **0.68 a 0.92**. El único fallo restante —una consulta sobre «modelo relacional» que no trae «Bases de Datos» al top-5, aun cuando su ficha sí menciona el modelo relacional— expone el límite del *embedder* por defecto (MiniLM, chico y centrado en inglés) sobre vocabulario técnico en español. **Lo verificamos**: al reemplazar el embedder por uno multilingüe (*multilingual-e5-large*), la recuperación sube a **hit-rate@1 = 1.00** y **MRR = 1.00** —las 12 consultas, «Bases de Datos» incluida, recuperan la ficha correcta en primer lugar—. Mantenemos MiniLM como *default* por su huella liviana y offline (≈90 MB frente a 2,2 GB); el multilingüe queda como mejora opcional ya validada, no como una promesa (reproducibile con `scripts/eval_retriever_e5.py`).

1.0	0.75	0.69	0.60
precisión factual CON RAG: pares (materia, nota) correctos (carta de pasantía)	/ 0.25 faithfulness (juez LLM) con / sin RAG	/ 0.22 grounding léxico con / sin RAG	/ 0.53 grounding semántico con / sin RAG

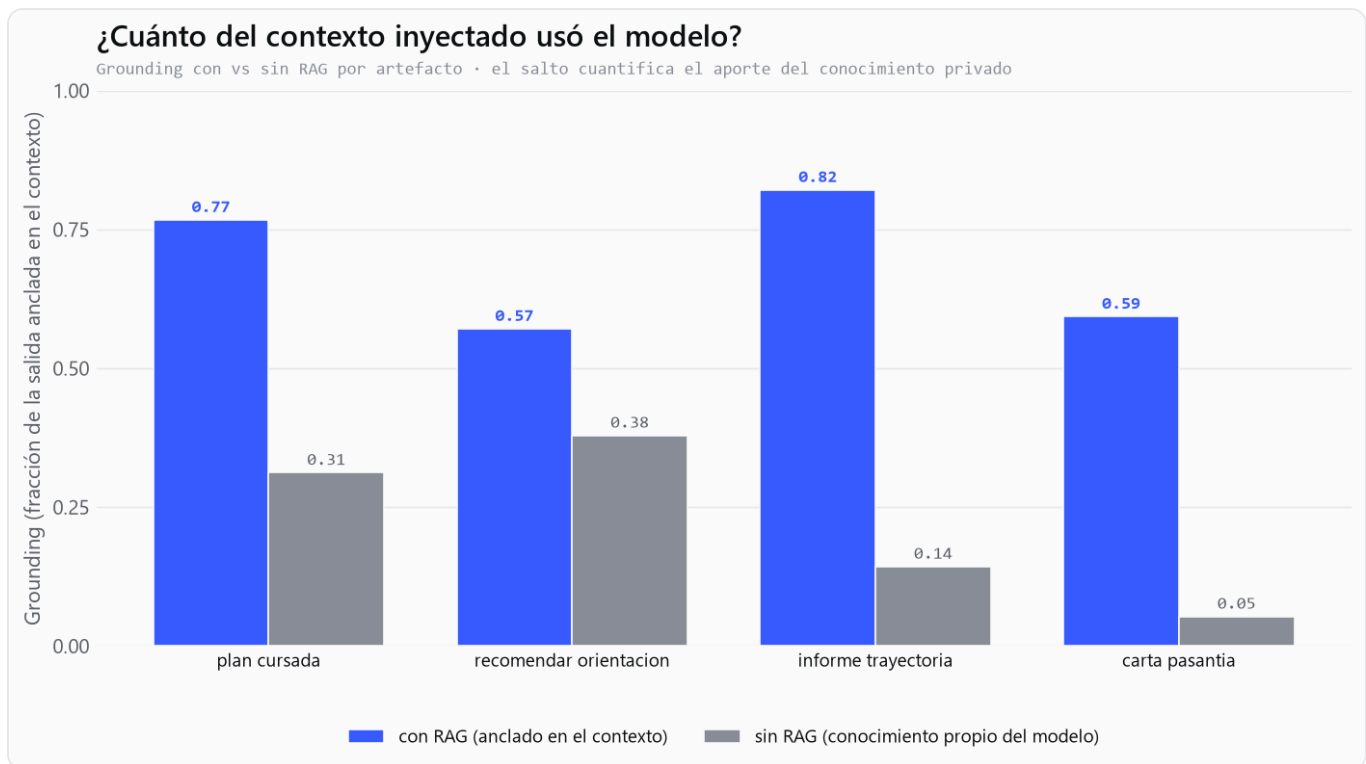


Fig. 6 – Cuánto del contexto inyectado usó el modelo: grounding **con** vs **sin** RAG por artefacto. El salto con-sin cuantifica el aporte del conocimiento privado (lo que el modelo no podía resolver con su conocimiento propio).

06 Análisis crítico y conclusiones

El sistema cumple el objetivo: construimos el pipeline RAG completo (no una caja negra) y lo usamos para **generar**, no para responder. La demo con/sin RAG y el grounding hacen visible y medible el valor del conocimiento privado.

Limitaciones

El grounding, aun en su versión semántica, no detecta *contradicciones* sutiles: mide cercanía con el contexto, no consistencia lógica con él; mide densidad de datos citados, no corrección del razonamiento. El recomendador ya cruza los **contenidos mínimos** de las materias (no solo sus nombres), pero las **electivas** no tienen ficha individual en el diseño curricular, así que sobre ellas se razona a nivel de régimen. Y la generación, anclada en el contexto y acotada por el lookup exacto de la base relacional, puede aun así deslizar matices imprecisos que solo una verificación de consistencia detectaría.

Trabajo futuro

Sumar las descripciones de las electivas (no detalladas en la ordenanza) para recomendarlas por contenido como ya se hace con las obligatorias; extender la evaluación del retriever —hoy con *hit-rate* y *MRR* sobre un gold set de 12 consultas— a un conjunto mayor, sumar *answer/context relevancy* con un LLM como juez y una verificación de consistencia lógica que ninguna métrica actual captura, e **integrar opcionalmente el embedder multilingüe** (e5-large, ya validado con recuperación perfecta) detrás de una bandera de configuración, para no imponer su descarga de 2,2 GB en el camino por defecto; y consolidar la arquitectura híbrida (modelo grande en GPU para la demo, base local reproducible para producción).

El código, el notebook integrador y la web app de demostración en vivo acompañan este informe como entregables ejecutables.